

Sistemas de Inteligencia Artificial

Trabajo Práctico N° 1: Métodos de Búsqueda



8-Puzzle



Sokoban



Grupo 9

Chiatellino, Máximo - 63477

Coleur, Matías - 63461

Curi Martínez, Gonzalo Sharif - 63463

Novillo, Valentina - 63156

Taurian, Magdalena - 62828

8-Puzzle: Esencia del Juego

01. 8-PUZZLE

🧩 Mecánica Principal

Juego de tablero deslizable compuesto por una grilla de **3×3** con **8 fichas numeradas (1 a 8)** y **1 espacio vacío (0)**.

🎯 **Objetivo:** Alcanzar el estado meta moviendo las fichas adyacentes al espacio vacío.

👤 **Acciones:** Mover el espacio {ARRIBA, ABAJO, IZQUIERDA, DERECHA}.

Ejemplo tablero inicial:

5	7	3
8	2	
1	6	4

Tableros solución:

1	2	3
4	5	6
7	8	

3	2	1
6	5	4
	8	7

3	6	
2	5	8
1	4	7

</> Representación Elegida

Modelado del tablero mediante una **Tupla Inmutable de 9 Elementos** (Aplanamiento de matriz 3×3).

5	7	3
8	2	0
1	6	4

Esta matriz se representaría como la tupla:

(5, 7, 3, 8, 2, 0, 1, 6, 4)

1. Distancia Manhattan

Suma de las distancias horizontales y verticales de cada ficha desde su posición actual hasta su posición objetivo.

Admisibilidad

No Trivialidad

Dominancia

2. Fichas Mal Ubicadas

Cantidad de fichas que no se encuentran en su casilla correspondiente respecto al estado objetivo.

Admisibilidad

No Trivialidad

8-Puzzle: Heurísticas – Ejemplos

1. Distancia Manhattan

5	7	3
8	2	
1	6	4

Estado n

$$h1(n) = 2+3+0+2+1+2+2+3 = 15$$

2. Fichas Mal Ubicadas

5	7	3
8	2	
1	6	4

Estado n

$$h2(n) = 8$$

1	2	3
4	5	6
7	8	

Estado solución

⚙️ ¿En qué consiste?

Búsqueda informada que evalúa los nodos según:

$$f(n) = g(n) + h(n)$$

¿Por qué elegimos A*?

Garantía de Solución Óptima

Filtro por paridad

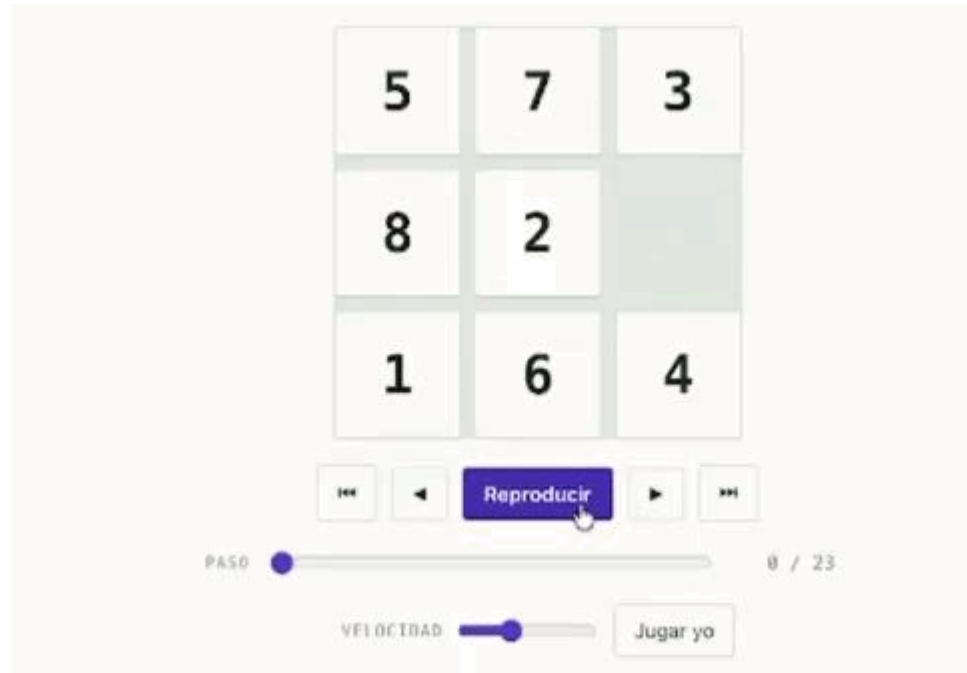
Para reducir el espacio de estados.

⚖️ Comparación Rápida

- ✓ **Vs. BFS:** A* expande menos nodos
- ✓ **Vs. DFS:** DFS no garantiza optimalidad
- ✓ **Vs. Greedy:** Greedy es más rápido pero no óptimo

8-Puzzle: DEMO A* Manhattan

01.8-PUZZLE



Reglas y Mecánicas

Empuje único → una caja por acción, hereda la dirección del jugador

Paredes/objetivos → estáticos



Estructura Dinámica

Objeto con dos campos: posición del jugador (una coordenada) y la de las cajas (conjunto de coordenadas):

```
Coord = tuple[int, int]          # una coordenada (x, y)

@dataclass(frozen=True, slots=True)
class State:
    player: Coord                # UNA sola coordenada: (x, y)
    boxes: frozenset[Coord]      # un CONJUNTO de coordenadas
```

1. Distancia Manhattan + Bipartite Matching

Como las cajas son intercambiables, primero se decide qué caja va a qué objetivo. Se arma la tabla de distancias Manhattan ($|\Delta x| + |\Delta y|$, en línea recta, ignorando paredes) caja $\rightarrow$ objetivo y se toma el emparejamiento de menor suma total, un objetivo por caja.

Admisibilidad

No Trivialidad

2. Distancia de Empuje Real (Real Push)

Misma estructura, pero la tabla usa la distancia real esquivando paredes, precalculada una vez por nivel con un BFS desde cada objetivo.

Admisibilidad

No Trivialidad

Dominancia

Algoritmo Hungaro

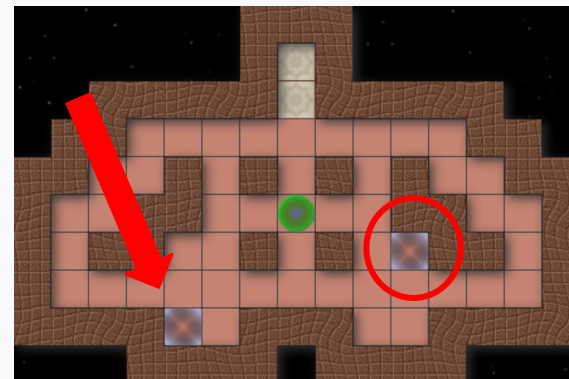
Implementación

- Arma una tabla con el costo de mandar cada caja a cada objetivo.
- Busca la combinación 1 a 1 que minimiza el total, probando de forma inteligente en vez de por fuerza bruta.
- Va asignando caja por caja y reacomoda asignaciones anteriores si conviene.

¿Por qué sirve?

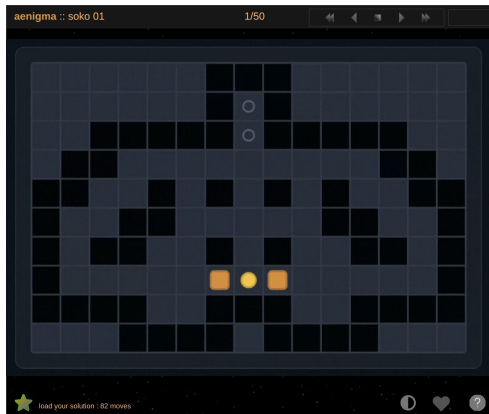
- Encuentra el óptimo en tiempo polinomial, no factorial ($O(\text{cajas}^2 * \text{goals})$ vs. $O(n!)$).
- El resultado es una cota inferior real del costo de la solución.

- Consideramos una caja bloqueada como un **GAME OVER**
- Reduce hasta casi la mitad de nodos expandidos con ciertos algoritmos



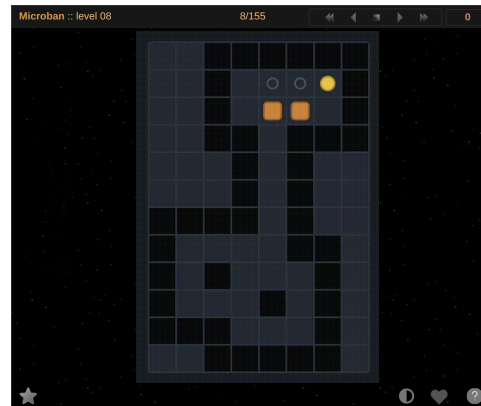
Sokoban: Tableros elegidos

02. SOKOBAN



aenigma_01.txt

Mapa mediano.
2 cajas
estrategia fácil

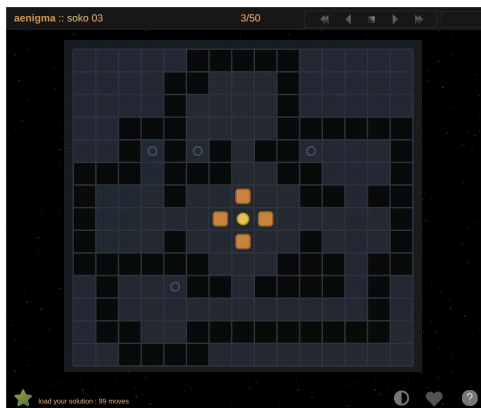


microban_08.txt

Mapa chico.
2 cajas
estrategia media

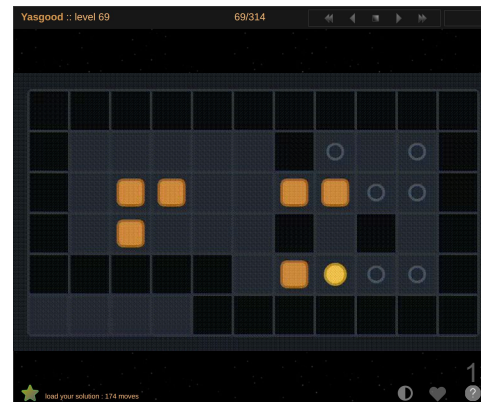
aenigma_03.txt

Mapa grande.
4 cajas
estrategia media



Yasgood_69.txt

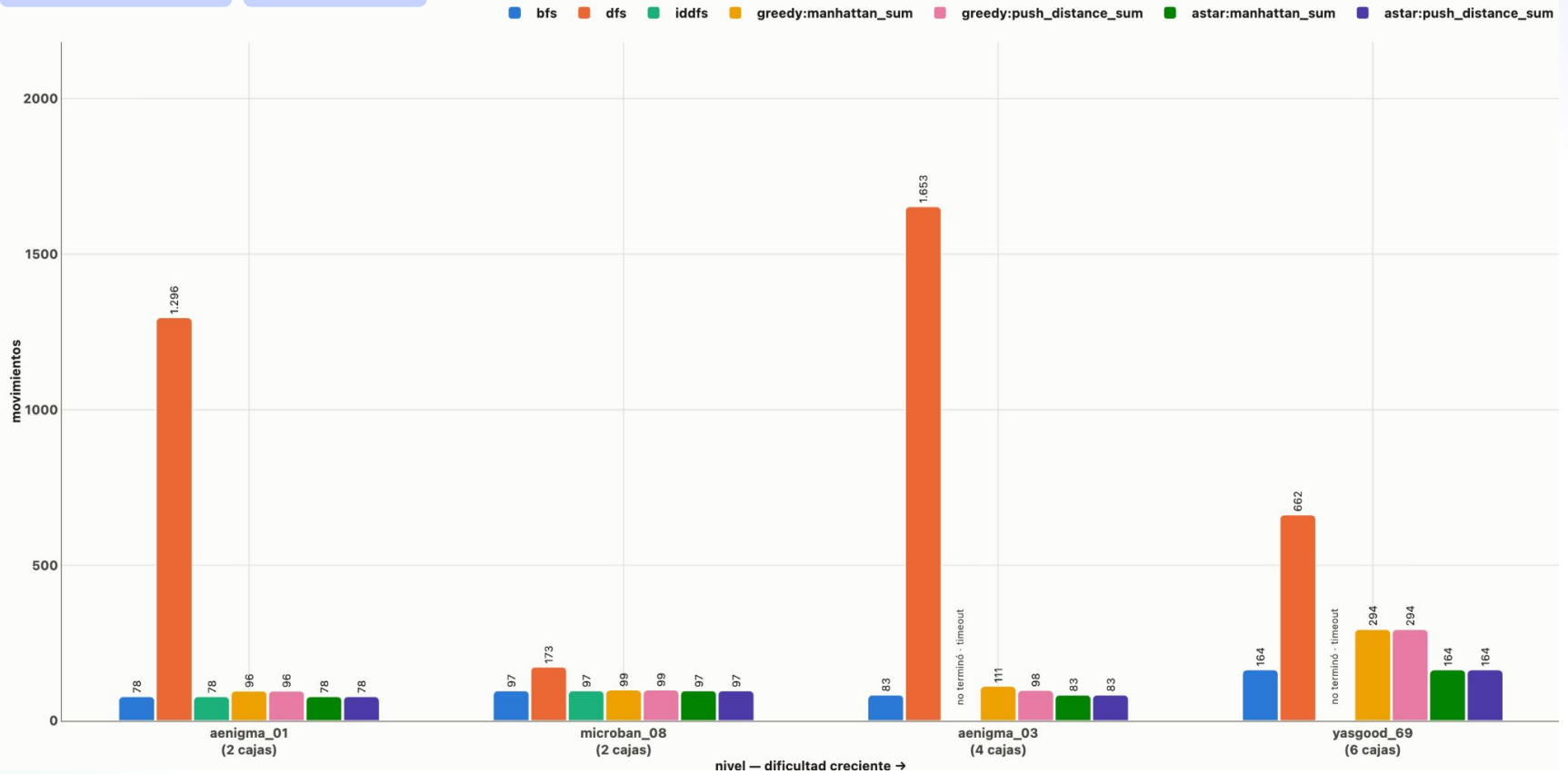
Mapa chico.
6 cajas
estrategia difícil



Costo de Solución por Nivel

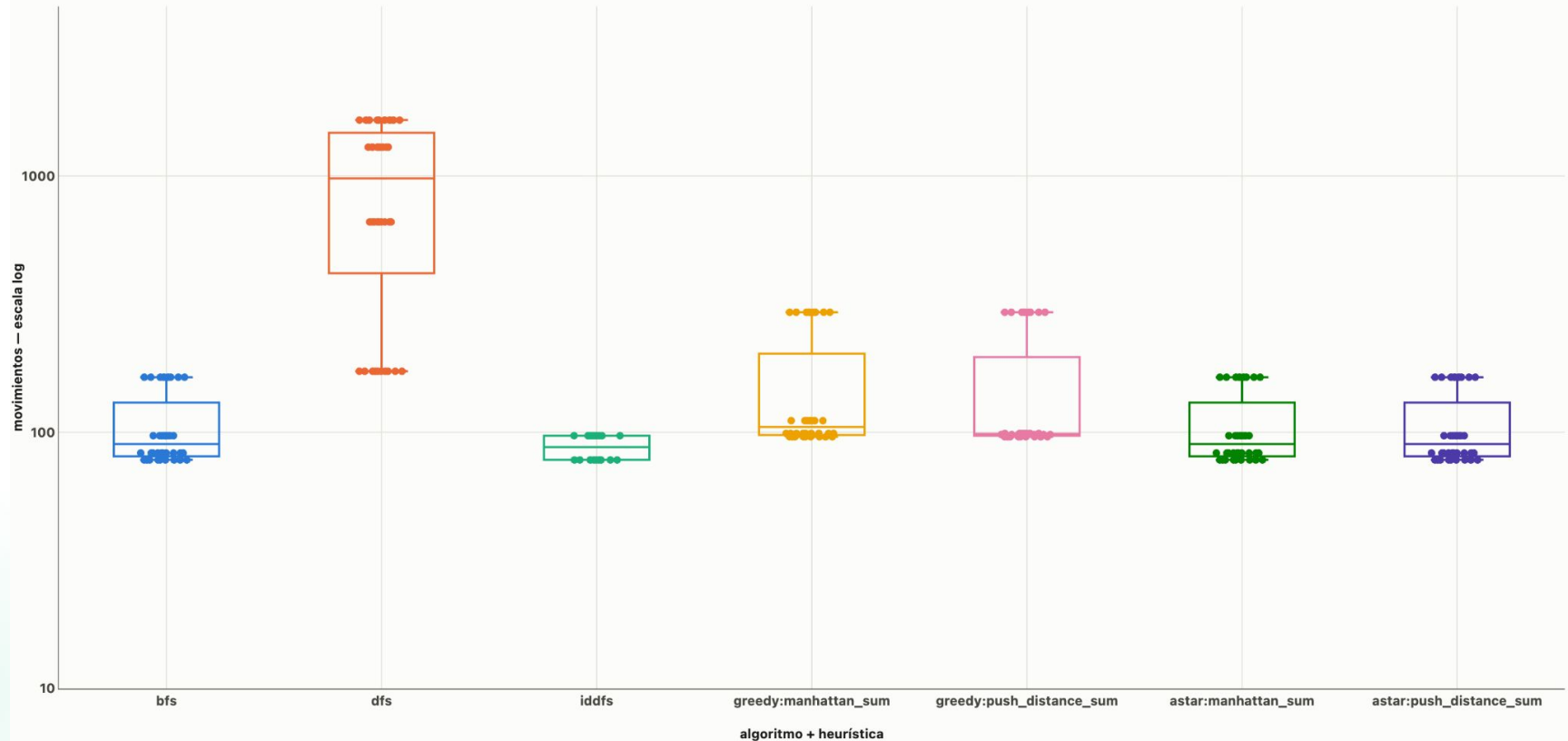
10 Repeticiones

30 Minutos



Dispersión de Costo por Algoritmo

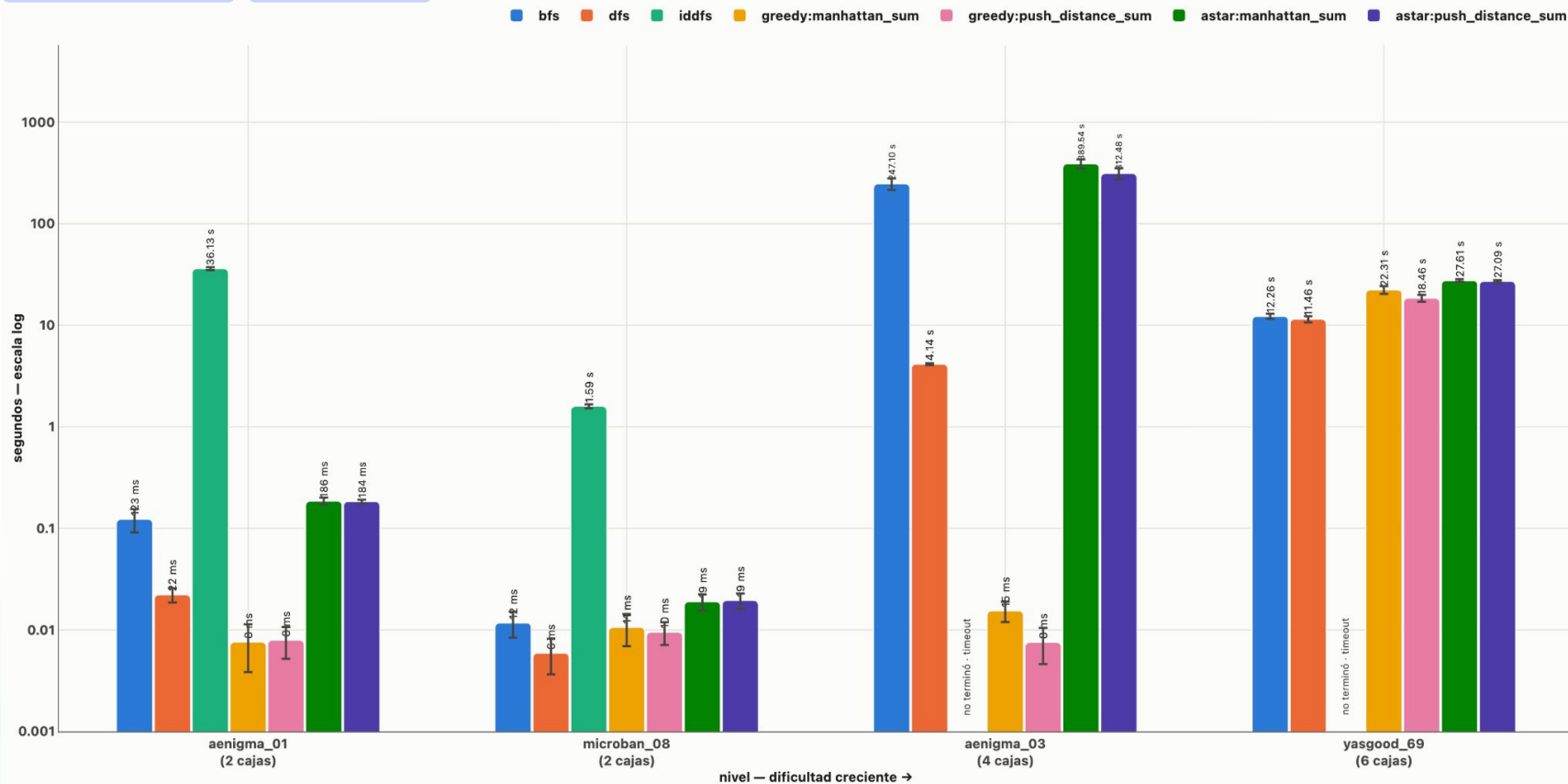
10 Repeticiones



Tiempo de Resolución por Nivel

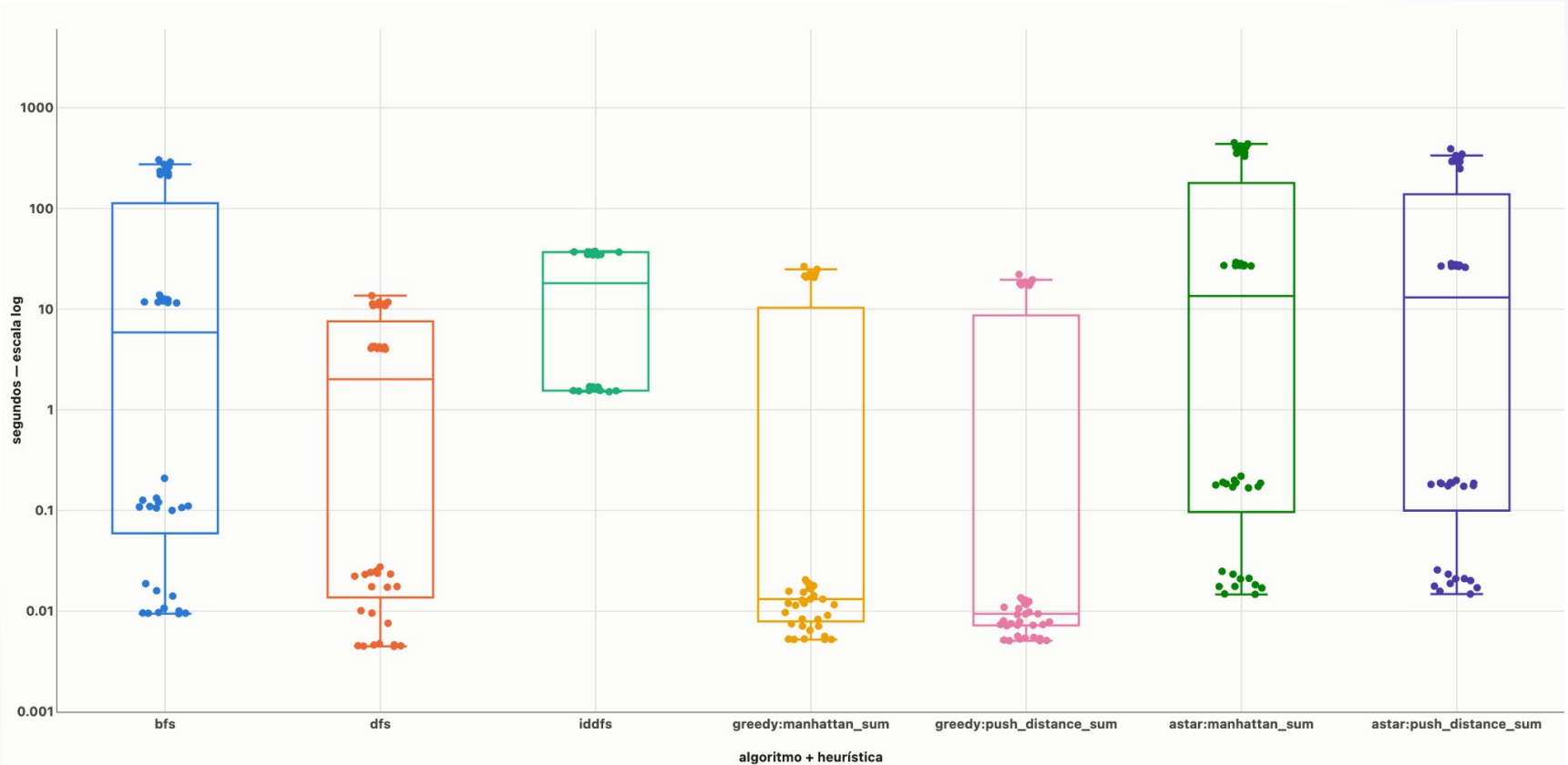
10 Repeticiones

30 Minutos



Dispersión de Tiempo por Algoritmo

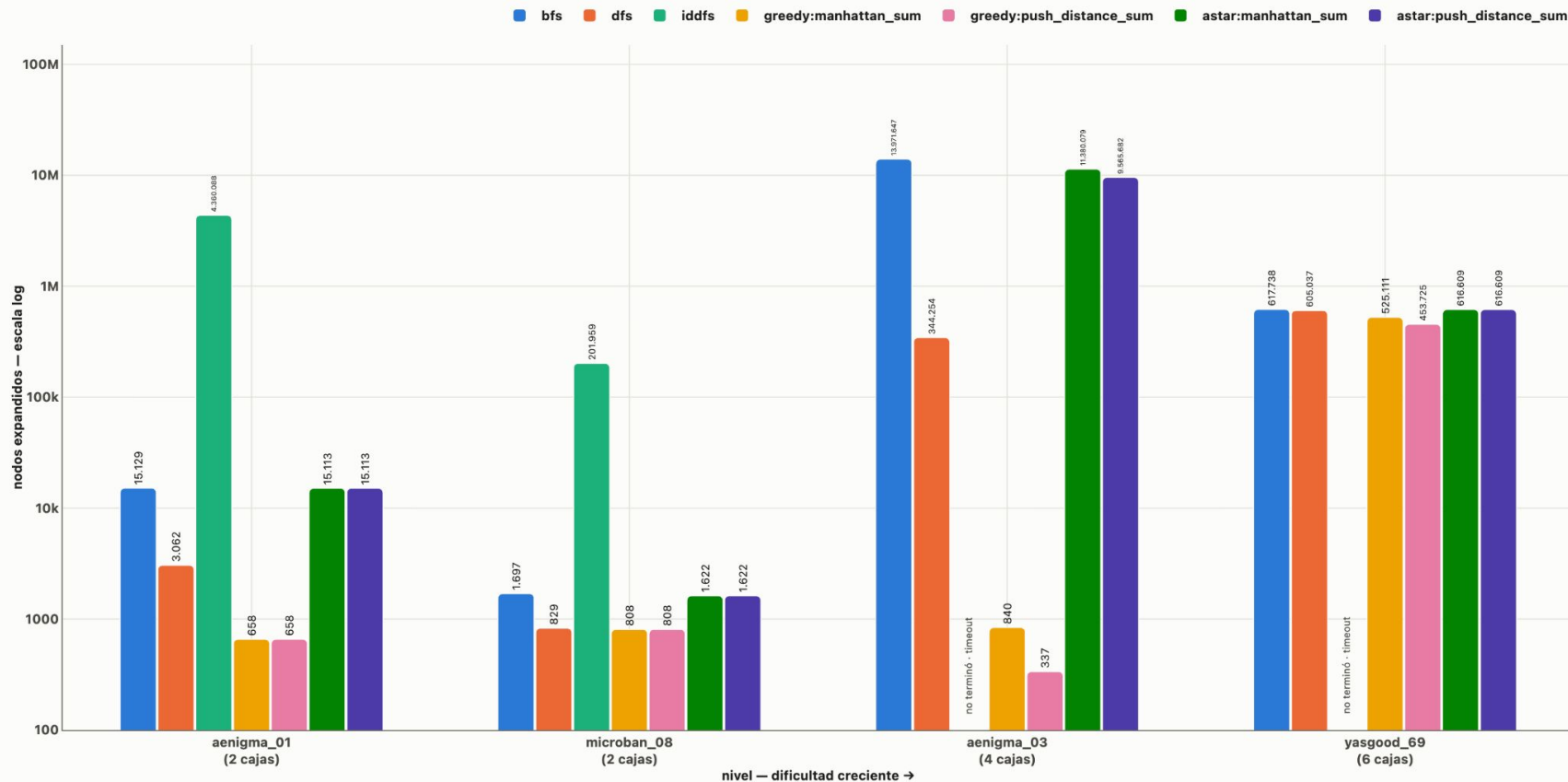
10 Repeticiones



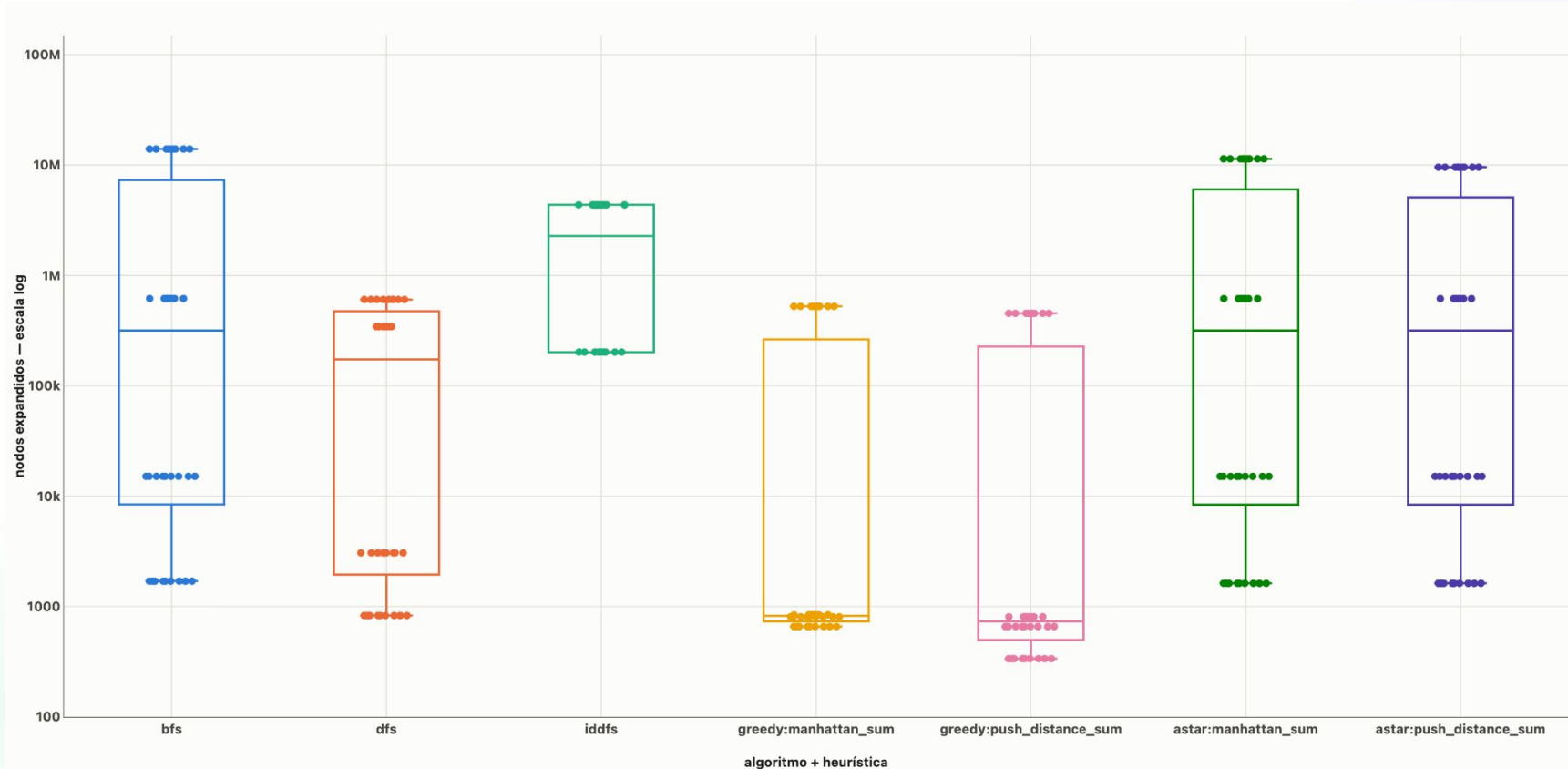
Nodos Expandidos por Nivel

10 Repeticiones

30 Minutos



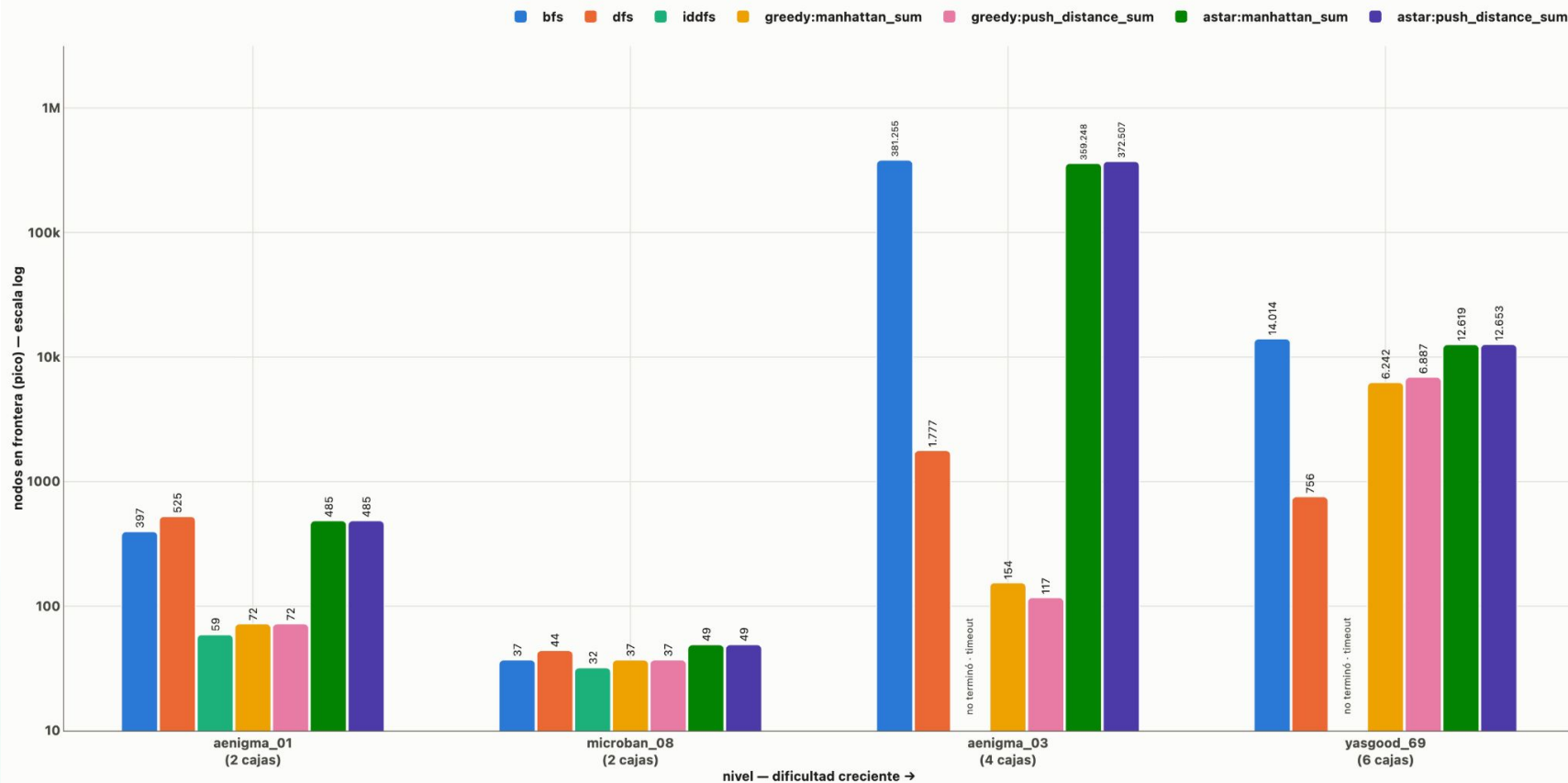
Dispersión de Nodos por Algoritmo



Frontera Máxima por Nivel

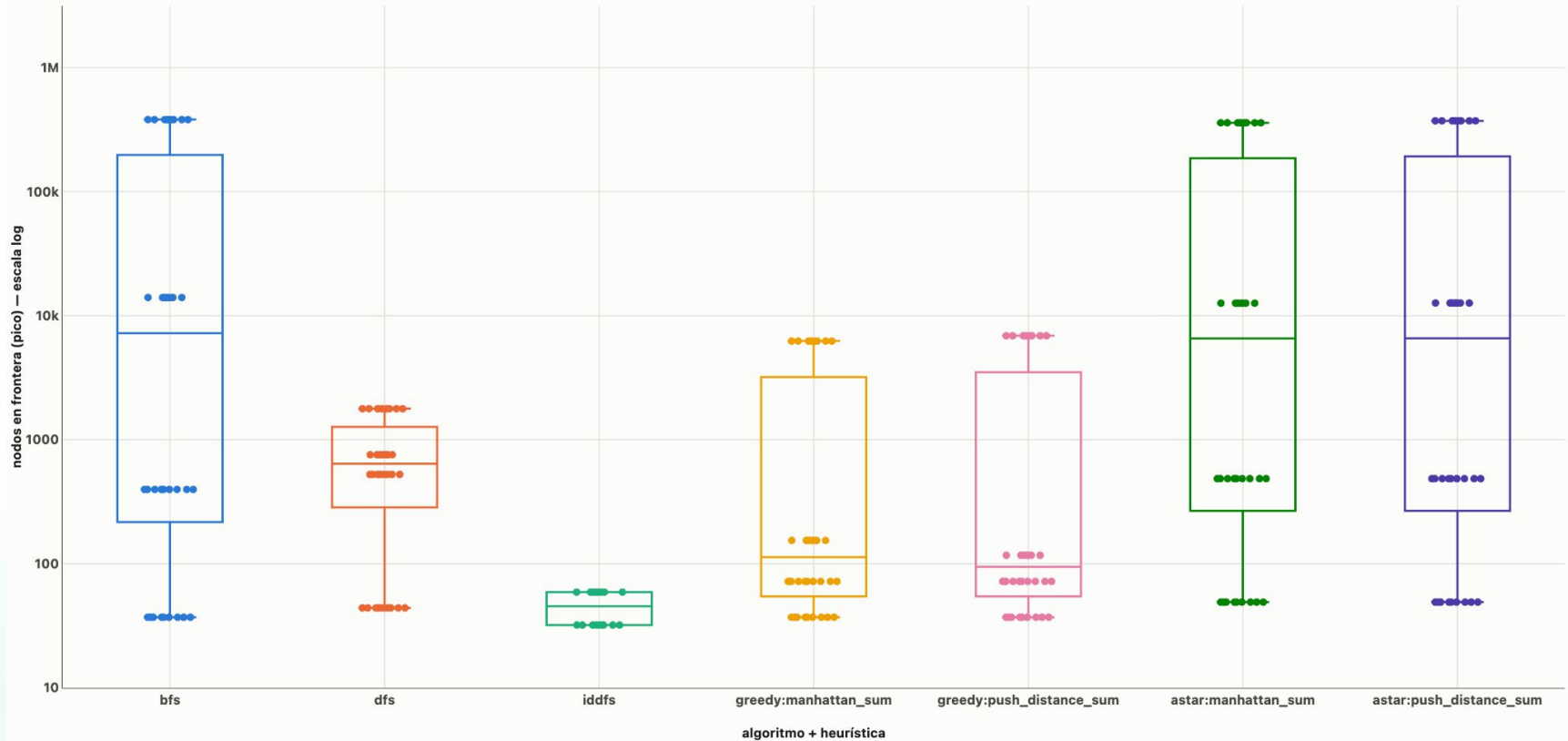
10 Repeticiones

30 Minutos



Dispersión de Frontera Máxima por Algoritmo

10 Repeticiones



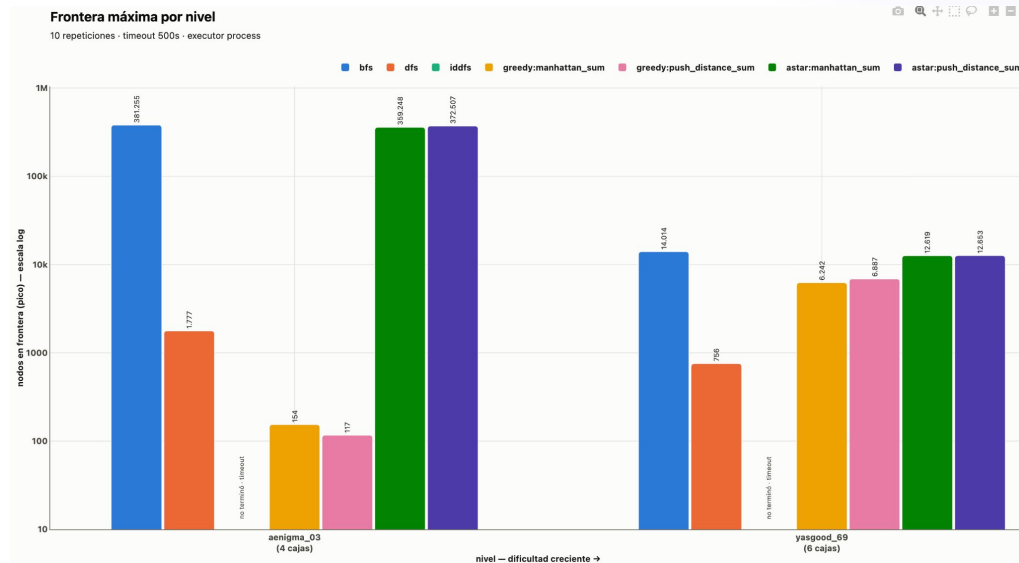
Conclusiones

1. Métodos desinformados: cada uno paga por un lado

- BFS → memoria
- DFS → soluciones malas
- IDDFS → tiempo
- En los difíciles, no terminó

2. Lo que los complica no es el tamaño

- Misma cantidad de cajas, costos muy distintos
- Pesa cómo está armado el tablero
- DFS encuentra algo rápido, pero malo



Conclusiones

3. A* no le ganó a BFS

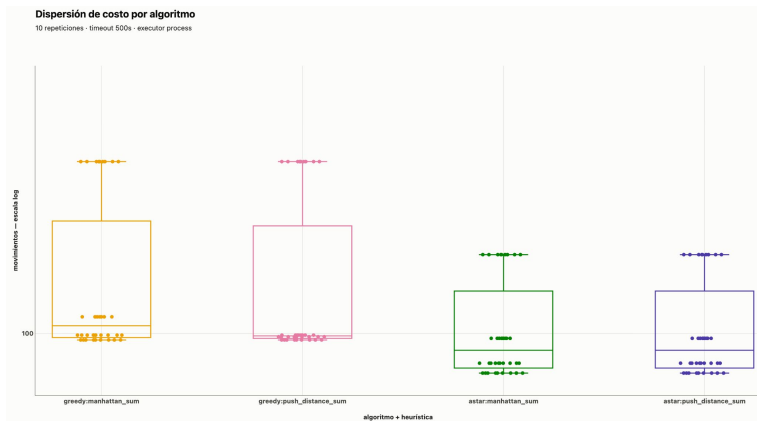
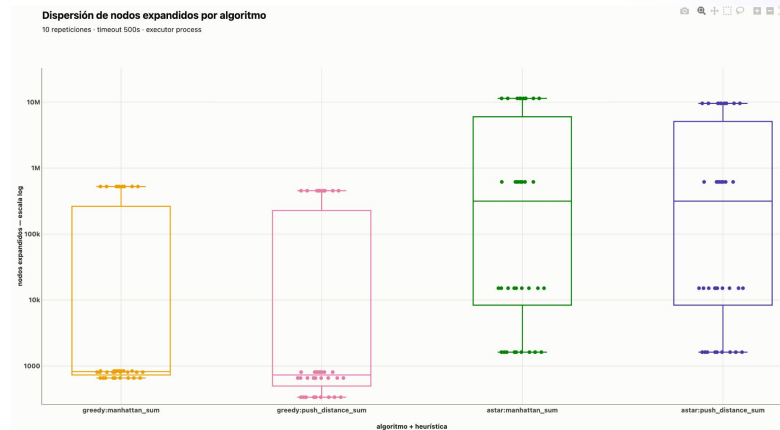
- Misma solución
- Apenas menos nodos
- Y tardó más

4. Greedy fue el más eficiente

- Expande poquísimo
- Soluciones buenas, no óptimas
- Con muchas cajas se le va el costo

5. La heurística importa más en Greedy

- En A*, cambiarla casi no movió nada
- En Greedy sí
- En A* manda el costo acumulado



¿Cuál es mejor?

Conclusiones

6. No hay uno mejor

- Óptimo y sobra memoria → **BFS**
- Óptimo con poca memoria → **IDDFS**
- Rápido y alcanza con buena → **Greedy**
- A* necesita mejor heurística



El famoso DEPENDE



GRACIAS

Trabajo Práctico N° 1: Métodos de Búsqueda



8-Puzzle



Sokoban



Grupo 9